Multiple Android Activities

I. Application Architecture Context

- **Contextual Application:** Rivi (Minimalist Academic Microlearning)
- **Architectural Driver:** Model-View-ViewModel (MVVM) Design Pattern
- **Core Features Elaborated:** Real-world educational applications heavily rely on segregating user workflows into distinct spatial sandboxes. By distributing the user-flow across multiple `Activities`, the application naturally inherits built-in back-stack management handled seamlessly by the OS. In this scope, we focus on engineering the explicit mapping between a global navigation dashboard surface and a hyper-focused study interface, bridging them through the strategic dispatch of `Intents` bearing bundled payload data (such as primitive subject identification keys).

II. Activity Phase 1: Dashboard Interface (MainActivity)

The `MainActivity` operates as the singular entry gateway (defined as `ACTION_MAIN` and `CATEGORY_LAUNCHER` in the manifest). It is structurally responsible for orchestrating global user progression.

- **Functional Elaboration:** The primary operational logic within this activity involves establishing an observable connection to the local Room database via its ViewModel. It retrieves the saved subject matrices asynchronously on background IO threads to prevent UI-stutter, rendering them via a customized `RecyclerView` layout manager. When a user wishes to interact with a specific domain, the `MainActivity` intercepts the `onClick` event and synthesizes an `Explicit Intent`. It attaches the targeted `SubjectID` to the intent's extras bundle, subsequently initiating the hand-off procedure via `startActivity()`.

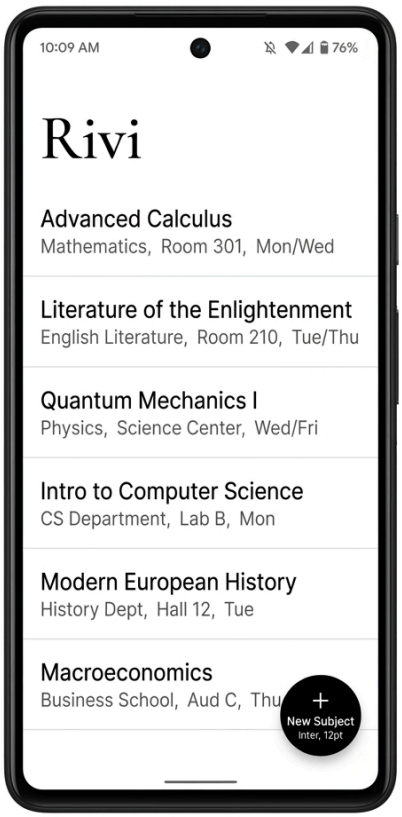

Figure 1: Cropped View of the Dashboard Operations

III. Activity Phase 2: Active Recall Protocol (StudySessionActivity)

The `StudySessionActivity` diverges significantly from the dashboard format. It abandons lists and structural navigation mechanics, embracing a highly constrained, distraction-free flashcard interface that fills the entire viewport matrix.

- **Functional Elaboration:** Upon initialization (`onCreate()`), this activity instantly unpacks the incoming `Intent` bundle to securely extract the targeted `SubjectID`. Instead of loading all data globally, it exclusively queries the persistence layer for flashcards localized entirely into the "Smart 5" daily review limits. It then manages the localized feedback loops—capturing user metric inputs (Again, Hard, Good, Easy) and dispatching them structurally back through the ViewModel to update the specific Spaced Repetition (SM-2) scheduling integers within the local store.

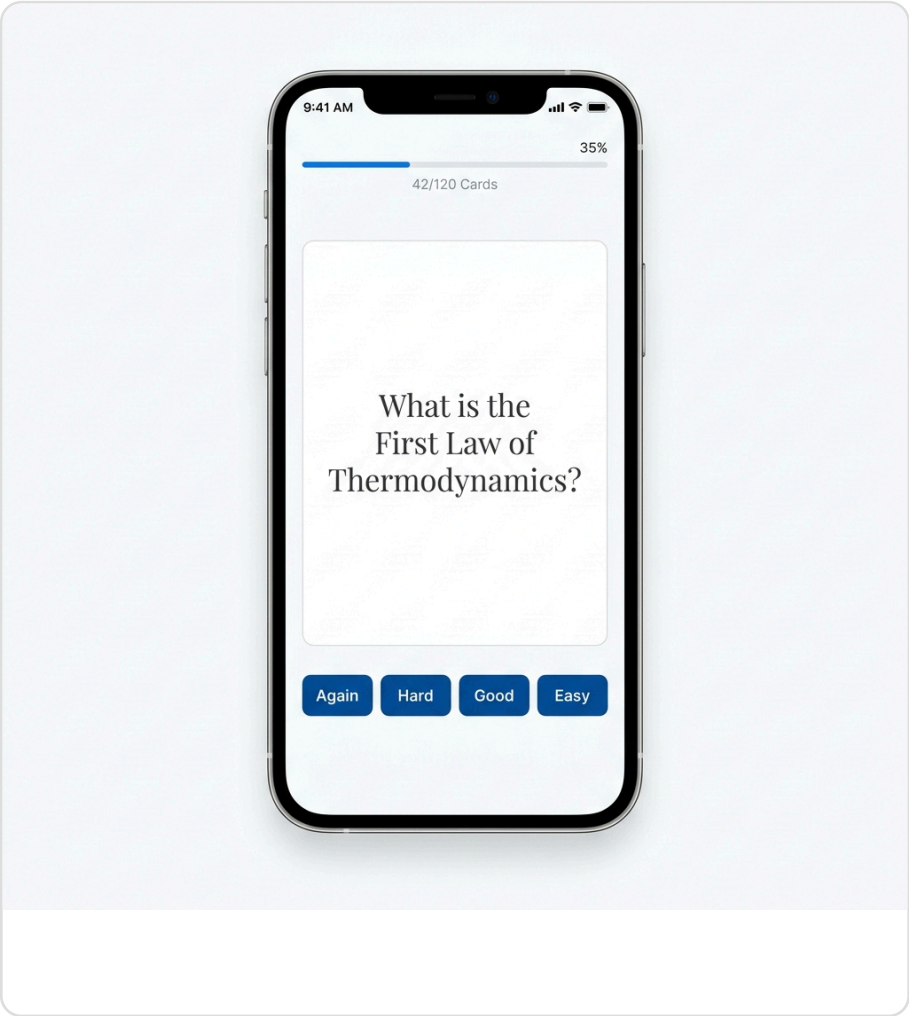

Figure 2: Cropped View of the Distraction-Free Flashcard Mode

IV. Activity Phase 3: Content Administration (CardManagementActivity)

The `CardManagementActivity` is the operational interface specifically crafted to facilitate Create, Read, Update, and Delete (CRUD) dynamics for the application's core knowledge graphs.

- **Functional Elaboration:** This component is instantiated via an `Intent` that carries necessary hierarchical parameters (i.e. to which Subject these flashcards belong). It provisions a scrolling linear layout mapped inversely to the database via Room DAOs. It supports real-time form inputs, allowing users to modify existing cards or inject new learning material directly into the local SQLite persistence layer which instantly synchronizes with the `MainActivity` state upon finish.

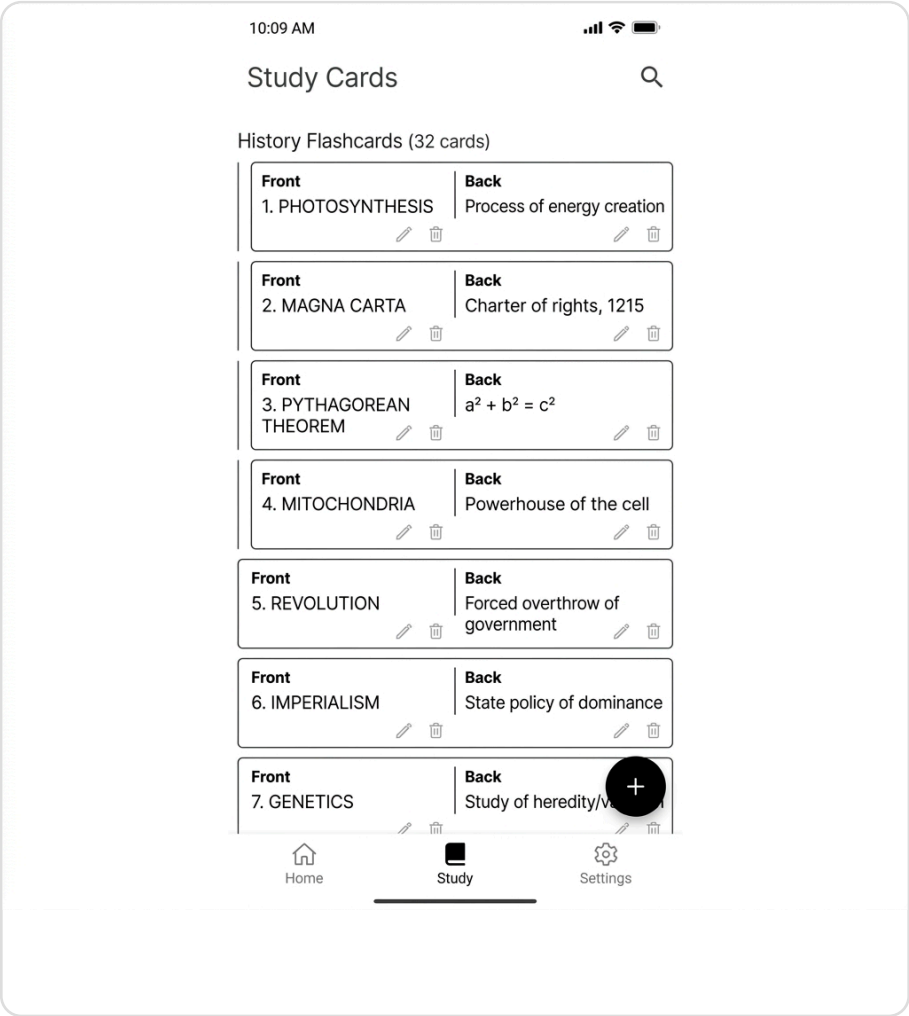

Figure 3: Cropped View of the Card Management Matrix

V. Activity Phase 4: Session Analytics (SessionRecapActivity)

Following the completion of an active recall phase, the navigation flow inherently delegates control dynamically to the `SessionRecapActivity`.

- **Functional Elaboration:** Instead of abruptly terminating user engagement, this activity bridges the conclusion of studying and returning to the dashboard. The `StudySessionActivity` invokes this via an explicit parameter launch, passing session accuracy metrics inside the Intent bundle. The `SessionRecapActivity` extracts these parameters formatting them into elegant progress telemetry (e.g. studying intensity, percentage accuracy) reinforcing learning behaviors before safely popping the activity stack and routing the user safely back home.

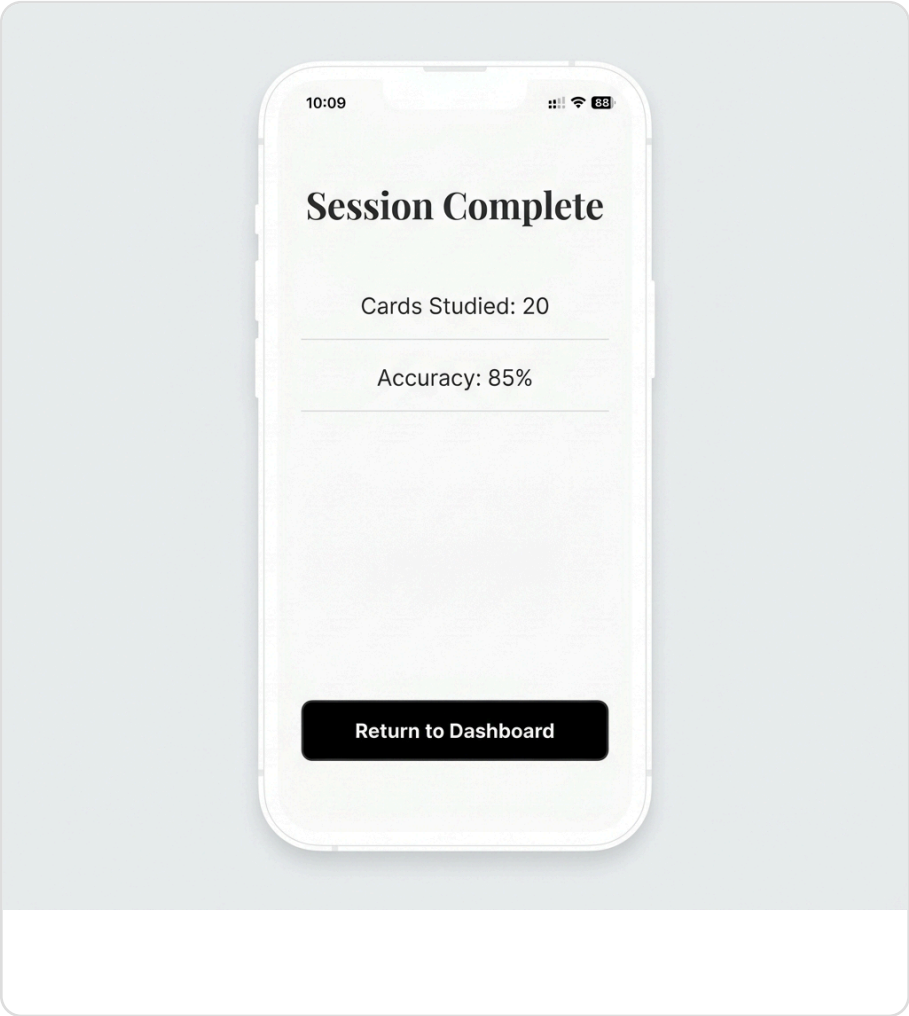

Figure 4: Cropped View of the Analytics Summary

VI. Artifact 1: Core Dashboard Implementation

Below is the structured Java architecture that bounds the UI logic for `MainActivity.java`.

```java
package me.prasad.study_app;

import android.app.DatePickerDialog;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.animation.AnimationUtils;
import android.view.animation.LayoutAnimationController;
import android.widget.ImageButton;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.button.MaterialButton;
import com.google.android.material.floatingactionbutton.ExtendedFloatingActionButton;
import com.google.android.material.textfield.TextInputEditText;

import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.Locale;

import me.prasad.study_app.data.entity.Subject;
import me.prasad.study_app.ui.CardManagementActivity;
import me.prasad.study_app.ui.StudySessionActivity;
import me.prasad.study_app.ui.adapter.SubjectAdapter;
import me.prasad.study_app.viewmodel.SubjectViewModel;

public class MainActivity extends AppCompatActivity {

    private final Calendar calendar = Calendar.getInstance();
    private SubjectViewModel viewModel;
    private final ActivityResultLauncher<String> importLauncher =
registerForActivityResult(
            new ActivityResultContracts.GetContent(),
            uri -> {
                if (uri != null) {
                    importSubject(uri);
                }
            }
    );
```

```
    private SubjectAdapter adapter;
    private int pendingExportSubjectId = -1;
    private final ActivityResultLauncher<String> exportLauncher =
registerForActivityResult(
            new ActivityResultContracts.CreateDocument("application/json"),
            uri -> {
                if (uri != null) {
                    exportSubject(uri);
                }
            }
    );

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        setupRecyclerView();

        viewModel = new ViewModelProvider(this).get(SubjectViewModel.class);
        viewModel.getAllSubjects().observe(this, subjects ->
adapter.submitList(subjects));

        ExtendedFloatingActionButton fab = findViewById(R.id.fab_add_subject);
        fab.setOnClickListener(v -> showAddSubjectDialog());

        ImageButton btnImport = findViewById(R.id.btn_import);
        btnImport.setOnClickListener(v -> importLauncher.launch("application/json"));
    }

    private void importSubject(Uri uri) {
        try {
            viewModel.importSubject(getContentResolver().openInputStream(uri), () ->
runOnUiThread(() -> Toast.makeText(this, "Subject imported",
Toast.LENGTH_SHORT).show()));
        } catch (Exception e) {
            Toast.makeText(this, "Import failed", Toast.LENGTH_SHORT).show();
        }
    }

    private void exportSubject(Uri uri) {
        if (pendingExportSubjectId == -1) return;
        try {
            viewModel.exportSubject(pendingExportSubjectId,
getContentResolver().openOutputStream(uri));
            Toast.makeText(this, "Exporting...", Toast.LENGTH_SHORT).show();
            pendingExportSubjectId = -1;
        } catch (Exception e) {
            Toast.makeText(this, "Export failed", Toast.LENGTH_SHORT).show();
        }
    }
```

```
    private void showAddSubjectDialog() {
        View view = LayoutInflater.from(this).inflate(R.layout.dialog_add_subject, null);
        TextInputEditText nameInput = view.findViewById(R.id.edit_subject_name);
        TextInputEditText dateInput = view.findViewById(R.id.edit_exam_date);
        MaterialButton btnSave = view.findViewById(R.id.btn_save);
        MaterialButton btnCancel = view.findViewById(R.id.btn_cancel);

        AlertDialog dialog = new AlertDialog.Builder(this)
                .setView(view)
                .create();

        dateInput.setOnClickListener(v -> new DatePickerDialog(this, (view1, year, month,
dayOfMonth) -> {
            calendar.set(Calendar.YEAR, year);
            calendar.set(Calendar.MONTH, month);
            calendar.set(Calendar.DAY_OF_MONTH, dayOfMonth);
            SimpleDateFormat sdf = new SimpleDateFormat("MMM dd, yyyy",
Locale.getDefault());
            dateInput.setText(sdf.format(calendar.getTime()));
        }, calendar.get(Calendar.YEAR), calendar.get(Calendar.MONTH),
calendar.get(Calendar.DAY_OF_MONTH)).show());

        btnSave.setOnClickListener(v -> {
            if (nameInput.getText() == null || dateInput.getText() == null) return;

            String name = nameInput.getText().toString().trim();
            if (name.isEmpty() || dateInput.getText().toString().isEmpty()) {
                Toast.makeText(this, "Please fill all fields",
Toast.LENGTH_SHORT).show();
                return;
            }

            Subject subject = new Subject(name, calendar.getTimeInMillis(), 0);
            viewModel.insert(subject);
            dialog.dismiss();
        });

        btnCancel.setOnClickListener(v -> dialog.dismiss());

        dialog.show();
    }

    private void setupRecyclerView() {
        RecyclerView recyclerView = findViewById(R.id.recycler_subjects);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));

        LayoutAnimationController animation = AnimationUtils.loadLayoutAnimation(this,
R.anim.layout_slide_up);
        recyclerView.setLayoutAnimation(animation);
        adapter = new SubjectAdapter(new
ViewModelProvider(this).get(SubjectViewModel.class), this);
        recyclerView.setAdapter(adapter);
```

```
        adapter.setOnSubjectClickListener(new SubjectAdapter.OnSubjectClickListener() {
            @Override
            public void onSubjectClick(Subject subject) {
                Intent intent = new Intent(MainActivity.this,
StudySessionActivity.class);
                intent.putExtra(StudySessionActivity.EXTRA_SUBJECT_ID,
subject.getSubjectId());
                startActivity(intent);
            }

            @Override
            public void onManageCardsClick(Subject subject) {
                Intent intent = new Intent(MainActivity.this,
CardManagementActivity.class);
                intent.putExtra(CardManagementActivity.EXTRA_SUBJECT_ID,
subject.getSubjectId());
                startActivity(intent);
            }

            @Override
            public void onExportClick(Subject subject) {
                pendingExportSubjectId = subject.getSubjectId();
                String name = subject.getName();
                String fileName = (name != null ? name.replaceAll("\\s+", "_") :
"subject") + ".json";
                exportLauncher.launch(fileName);
            }
        });
    }
}
```

VII. Artifact 2: Active Recall Logic Implementation

Below is the structured Java architecture encompassing the UI operations within `StudySessionActivity.java`.

```java
package me.prasad.study_app.ui;

import android.animation.AnimatorInflater;
import android.animation.AnimatorSet;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.LinearLayout;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;
import androidx.lifecycle.ViewModelProvider;

import com.google.android.material.button.MaterialButton;
import com.google.android.material.card.MaterialCardView;

import java.util.List;

import me.prasad.study_app.R;
import me.prasad.study_app.data.entity.Flashcard;
import me.prasad.study_app.data.entity.Subject;
import me.prasad.study_app.logic.SRSLogic;
import me.prasad.study_app.viewmodel.FlashcardViewModel;
import me.prasad.study_app.viewmodel.SubjectViewModel;

public class StudySessionActivity extends AppCompatActivity {

    public static final String EXTRA_SUBJECT_ID = "extra_subject_id";

    private FlashcardViewModel flashcardViewModel;

    private TextView textSubjectTitle, textSessionProgress, textCardContent;
    private MaterialCardView cardFlashcard;
    private LinearLayout layoutGrading;

    private List<Flashcard> sessionCards;
    private Subject currentSubject;
    private boolean isShowingAnswer = false;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_study_session);

        int subjectId = getIntent().getIntExtra(EXTRA_SUBJECT_ID, -1);
        if (subjectId == -1) {
            finish();
```

```
            return;
        }

        initViews();
        setupViewModels(subjectId);
    }

    private void initViews() {
        textSubjectTitle = findViewById(R.id.text_subject_title);
        textSessionProgress = findViewById(R.id.text_session_progress);
        textCardContent = findViewById(R.id.text_card_content);
        cardFlashcard = findViewById(R.id.card_flashcard);
        layoutGrading = findViewById(R.id.layout_grading);

        MaterialButton btnAgain = findViewById(R.id.btn_again);
        MaterialButton btnHard = findViewById(R.id.btn_hard);
        MaterialButton btnGood = findViewById(R.id.btn_good);
        MaterialButton btnEasy = findViewById(R.id.btn_easy);

        cardFlashcard.setOnClickListener(v -> flipCard());

        btnAgain.setOnClickListener(v -> submitGrade(SRSLogic.Grade.AGAIN));
        btnHard.setOnClickListener(v -> submitGrade(SRSLogic.Grade.HARD));
        btnGood.setOnClickListener(v -> submitGrade(SRSLogic.Grade.GOOD));
        btnEasy.setOnClickListener(v -> submitGrade(SRSLogic.Grade.EASY));
    }

    private void setupViewModels(int subjectId) {
        SubjectViewModel subjectViewModel = new
ViewModelProvider(this).get(SubjectViewModel.class);
        flashcardViewModel = new ViewModelProvider(this).get(FlashcardViewModel.class);

        subjectViewModel.getSubjectById(subjectId).observe(this, subject -> {
            if (subject != null) {
                currentSubject = subject;
                String name = subject.getName();
                textSubjectTitle.setText(name != null ? name.toUpperCase() : "");
                flashcardViewModel.startSession(subject.getSubjectId(),
subject.getExamDate());
            }
        });

        flashcardViewModel.getSessionCards().observe(this, cards -> {
            sessionCards = cards;
            updateUI();
        });

        flashcardViewModel.getCurrentIndex().observe(this, index -> {
            isShowingAnswer = false;
            updateUI();
        });
```

```
        flashcardViewModel.getIsSessionComplete().observe(this, isComplete -> {
            if (isComplete) {
                Intent intent = new Intent(this, SessionRecapActivity.class);
                if (sessionCards != null) {
                    intent.putExtra("mastered_count", sessionCards.size());
                }
                startActivity(intent);
                finish();
            }
        });
    }

    private void flipCard() {
        if (sessionCards == null || sessionCards.isEmpty()) return;

        AnimatorSet flipOutSet = (AnimatorSet) AnimatorInflater.loadAnimator(this,
R.animator.card_flip_out);
        AnimatorSet flipInSet = (AnimatorSet) AnimatorInflater.loadAnimator(this,
R.animator.card_flip_in);

        flipOutSet.setTarget(cardFlashcard);
        flipInSet.setTarget(cardFlashcard);

        flipOutSet.addListener(new android.animation.AnimatorListenerAdapter() {
            @Override
            public void onAnimationEnd(android.animation.Animator animation) {
                isShowingAnswer = !isShowingAnswer;
                updateUI();
                flipInSet.start();
            }
        });
        flipOutSet.start();
    }

    private void updateUI() {
        Integer index = flashcardViewModel.getCurrentIndex().getValue();
        if (sessionCards == null || index == null || index >= sessionCards.size()) {
            return;
        }

        Flashcard currentCard = sessionCards.get(index);
        textSessionProgress.setText(getString(R.string.session_progress_format, (index +
1), sessionCards.size()));

        if (isShowingAnswer) {
            textCardContent.setText(currentCard.getAnswer());
            layoutGrading.setVisibility(View.VISIBLE);
            findViewById(R.id.text_tap_to_flip).setVisibility(View.INVISIBLE);
        } else {
            textCardContent.setText(currentCard.getQuestion());
            layoutGrading.setVisibility(View.GONE);
            findViewById(R.id.text_tap_to_flip).setVisibility(View.VISIBLE);
```

```
        }
    }

    private void submitGrade(SRSLogic.Grade grade) {
        Integer index = flashcardViewModel.getCurrentIndex().getValue();
        if (sessionCards != null && index != null && index < sessionCards.size()) {
            flashcardViewModel.submitGrade(sessionCards.get(index), currentSubject,
grade);
        }
    }
}
```

VIII. Artifact 3: Card Administration Flow

Below is the Java architecture defining the `CardManagementActivity`.

```
package me.prasad.study_app.ui;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.Toast;

import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.button.MaterialButton;
import com.google.android.material.textfield.TextInputEditText;

import me.prasad.study_app.R;
import me.prasad.study_app.data.entity.Flashcard;
import me.prasad.study_app.ui.adapter.FlashcardAdapter;
import me.prasad.study_app.viewmodel.FlashcardViewModel;

public class CardManagementActivity extends AppCompatActivity {

    public static final String EXTRA_SUBJECT_ID = "extra_subject_id";

    private FlashcardViewModel viewModel;
    private FlashcardAdapter adapter;
    private int subjectId;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_card_management);

        subjectId = getIntent().getIntExtra(EXTRA_SUBJECT_ID, -1);
        if (subjectId == -1) {
            finish();
            return;
        }

        setupRecyclerView();

        viewModel = new ViewModelProvider(this).get(FlashcardViewModel.class);
        viewModel.getFlashcardsForSubject(subjectId).observe(this, cards ->
adapter.submitList(cards));
    }

    private void setupRecyclerView() {
```

```
        RecyclerView recyclerView = findViewById(R.id.recycler_flashcards);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        adapter = new FlashcardAdapter();
        recyclerView.setAdapter(adapter);

        adapter.setOnCardDeleteListener(card -> new AlertDialog.Builder(this)
                .setTitle(R.string.delete_card)
                .setMessage("Are you sure you want to delete this card?")
                .setPositiveButton("Delete", (dialog, which) ->
viewModel.deleteFlashcard(card))
                .setNegativeButton("Cancel", null)
                .show());

        View fab = findViewById(R.id.fab_add_card);
        if (fab != null) {
            fab.setOnClickListener(v -> showAddCardDialog());
        }
    }

    private void showAddCardDialog() {
        View view = LayoutInflater.from(this).inflate(R.layout.dialog_add_card, null);
        TextInputEditText questionInput = view.findViewById(R.id.edit_question);
        TextInputEditText answerInput = view.findViewById(R.id.edit_answer);
        MaterialButton btnSave = view.findViewById(R.id.btn_save);
        MaterialButton btnCancel = view.findViewById(R.id.btn_cancel);

        AlertDialog dialog = new AlertDialog.Builder(this)
                .setView(view)
                .create();

        btnSave.setOnClickListener(v -> {
            String question = questionInput.getText() != null ?
questionInput.getText().toString().trim() : "";
            String answer = answerInput.getText() != null ?
answerInput.getText().toString().trim() : "";

            if (question.isEmpty() || answer.isEmpty()) {
                Toast.makeText(this, "Please fill all fields",
Toast.LENGTH_SHORT).show();
                return;
            }

            // New card with default SM-2 values
            Flashcard card = new Flashcard(subjectId, question, answer,
System.currentTimeMillis(), 0, 2.5f);
            viewModel.insertFlashcard(card);
            dialog.dismiss();
        });

        btnCancel.setOnClickListener(v -> dialog.dismiss());

        dialog.show();
```

```
    }
}
```

IX. Artifact 4: Session Analytics Handler

Below is the Java implementation controlling the `SessionRecapActivity` lifecycle.

```java
package me.prasad.study_app.ui;

import android.os.Bundle;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.google.android.material.button.MaterialButton;

import me.prasad.study_app.R;

public class SessionRecapActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_session_recap);

        int count = getIntent().getIntExtra("mastered_count", 0);
        TextView textSubtitle = findViewById(R.id.text_recap_subtitle);
        textSubtitle.setText(getString(R.string.recap_subtitle, count));

        MaterialButton btnFinish = findViewById(R.id.btn_finish);
        btnFinish.setOnClickListener(v -> finish());
    }
}
```